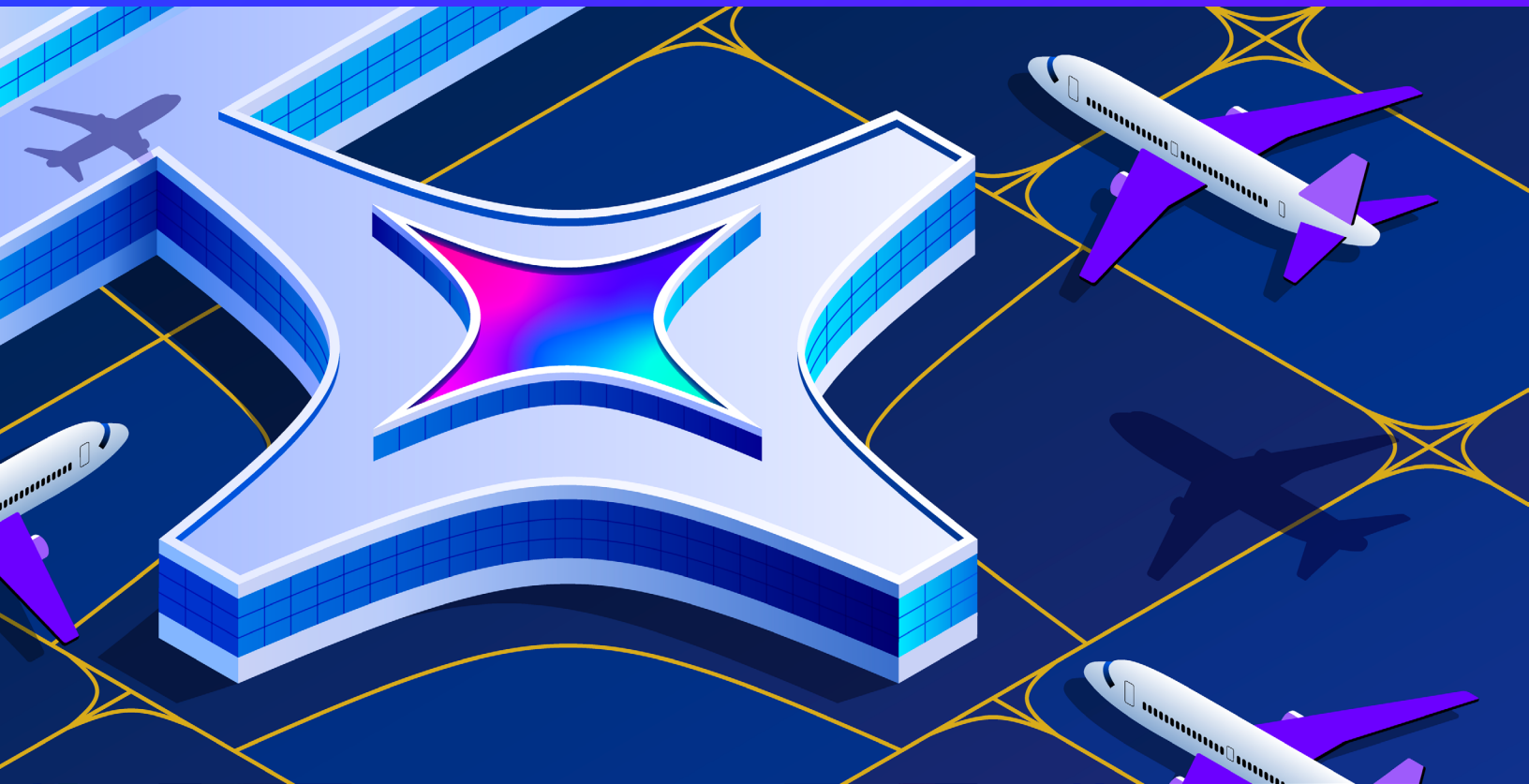


BEST PRACTICE SERIES

AI Security Best Practices Guide



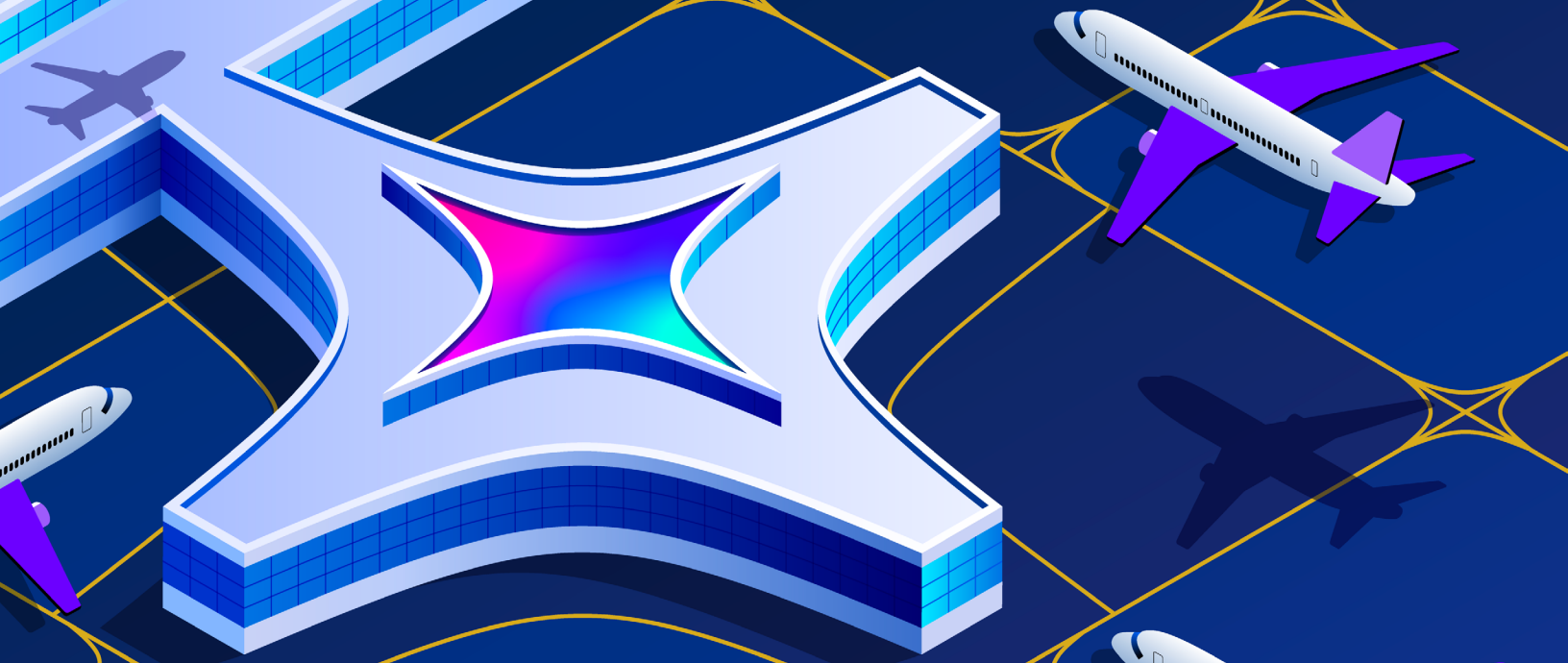
DATADOG

AI Security Best Practices Guide

Abusing AI infrastructure: How mismanaged credentials and resources expose LLM applications	3
Common attacks that target AI infrastructure	4
Detect and minimize the risks that lead to AI abuse	6
Minimize threats to AI infrastructure	8

Abusing supply chains: How poisoned models, data, and third-party libraries compromise AI systems	9
Common attacks that target AI artifacts in supply chains	9
Detect and minimize the inherent risks in supply chains	11
Minimize threats to AI artifacts in supply chains	13

Abusing AI interfaces: How prompt-level attacks exploit LLM applications	14
Common attacks that target AI interfaces	14
Embedding and executing harmful code	15
Exfiltrating data	16
Detect and minimize the inherent risks in AI interfaces	16
Protect your generative AI applications from abuse	18



Abusing AI infrastructure: How mismanaged credentials and resources expose LLM applications



Mallory Mooney

The swift adoption of generative AI (GenAI) by the software industry has introduced a new area of focus for security engineers: threats targeting the various components of their AI applications. Understanding how these areas are vulnerable to attacks will become increasingly significant as the space evolves. In this series, we'll look at common threats that target the following components of AI applications:

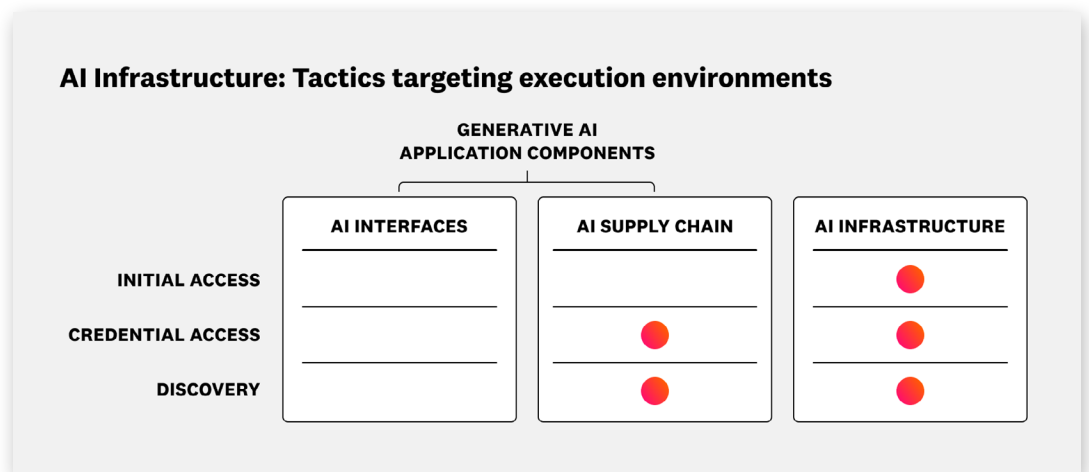
- **Infrastructure:** The underlying components that host and run AI applications, such as containers, databases, cloud storage buckets, and Model Context Protocol (MCP) servers
- **Supply chain:** The software and data that an AI application uses to operate, such as training datasets, pre-trained large language models (LLMs), and third-party AI libraries
- **AI interfaces:** The entry points and business logic that enable a user to interact with an AI application, such as chat interfaces and API endpoints for supporting services

We'll cover AI infrastructure in this chapter, mapping threats that target infrastructure components to [MITRE's Adversarial Threat Landscape for Artificial Intelligence Systems \(ATLAS\) tactics and techniques](#), which summarize why and how attackers target AI components. We'll also look at how to [detect and minimize these threats](#).

Common attacks that target AI infrastructure

The infrastructure for generative AI applications serves as a highway that facilitates scalable, reliable usage. It combines established cloud services with emerging tools and frameworks, such as [MCP servers](#) and retrieval-augmented generation (RAG) pipelines. Just like any transportation system, this infrastructure is vulnerable to a variety of risks. Common [cloud misconfigurations](#), such as overprivileged IAM roles, can be introduced at any time. And because the AI ecosystem is still maturing, its resources often launch without adequate controls for authentication, authorization, and logging.

Inefficient security controls and monitoring introduce vulnerabilities across an AI application's various components. The following diagram shows where certain tactics can take advantage of those weaknesses. For example, attackers often target AI infrastructure, including cloud user and service accounts, databases, and compute instances, both in the initial stages of their attack and as they move laterally after a compromise. Visualizing these connections helps you trace the path of an attack from end to end, including when they [acquire credentials](#) and [gather additional information](#) about resources.



GAINING INITIAL ACCESS

Initial access involves an attacker attempting to gain access to a particular AI system, such as parts of its infrastructure. There are several techniques that attackers use to accomplish this, including using [valid accounts](#) and [exploiting public-facing applications](#). These particular techniques are also a part of the [traditional MITRE ATT&CK framework](#) and take advantage of compromised user account credentials, API keys, or vulnerable infrastructure to gain access.

Let's look at an example of [gaining initial access to infrastructure by exploiting public-facing applications](#) and using prompt injection to poison an instance of M365 Copilot. In this example, the organization's Copilot

instance indexed all company emails in a retrieval augmented generation (RAG) database.

As part of a red teaming exercise, security researchers sent an employee an email that contained a hidden, malicious prompt and fraudulent bank account details. This step successfully exploited the company's public-facing email service to create a poisoned RAG entry. When certain employees queried the RAG database for financial information, the email service retrieved the poisoned entry and responded with the incorrect bank information. In the real world, using this information for a wire transfer could result in serious consequences for the company and the individuals who sent the query.

RAG systems in particular introduce new security risks to environments that are already prone to secret sprawl. By design, these systems index organizational data, such as knowledge repositories and codebases that may include sensitive information. If secrets are improperly managed or simply overlooked, RAG systems can unintentionally expose them in their output.

ACHIEVING CREDENTIAL ACCESS

Credential access is similar to initial access in that it uses credentials from legitimate user accounts to infiltrate an AI system. However, credential access involves actively attempting to find and steal credentials that are not stored securely within an environment.

Attackers were able to [use this tactic with Ray](#), a Python framework that is used for scaling AI applications. They successfully found a public-facing Ray cluster, which included exposed SSH keys, tokens, Kubernetes secrets, and cloud environment keys. These credentials allowed them to steal data and compromise other levels of the AI application, including its [supply chain](#).

Compromised credentials are the leading cause of cloud incidents. This risk grows as organizations integrate third-party libraries into their AI applications, which can be harder to audit than in-house code. If one of these libraries introduces a vulnerability, attackers can exploit it to find and compromise credentials before organizations can react.

GATHERING INFORMATION

Discovery involves an attacker using a variety of techniques to gather information about an AI system and its internal network after a compromise. Many discovery techniques target AI [supply chains](#) and [integrations](#), but the [cloud service discovery technique](#) attempts to gather information about services running in cloud infrastructure. These can

include enabled authentication, serverless, logging, and security services, as well as their associated policies and accounts.

LLM jacking is an example of how this technique shows up in an attack path. After attackers use stolen credentials to access a cloud environment, they look for available services that are running third-party LLMs, such as Amazon Bedrock. To do this, they execute legitimate API calls to these services to test their level of access without immediate detection. AWS CloudTrail has logged these calls, which has enabled researchers to identify atypical call parameters. In cases like these, where the discovery uses available APIs to probe a cloud environment, the value of logging service activity cannot be underestimated. Without this visibility, attackers would have continued to exploit their access to the cloud-hosted LLMs.

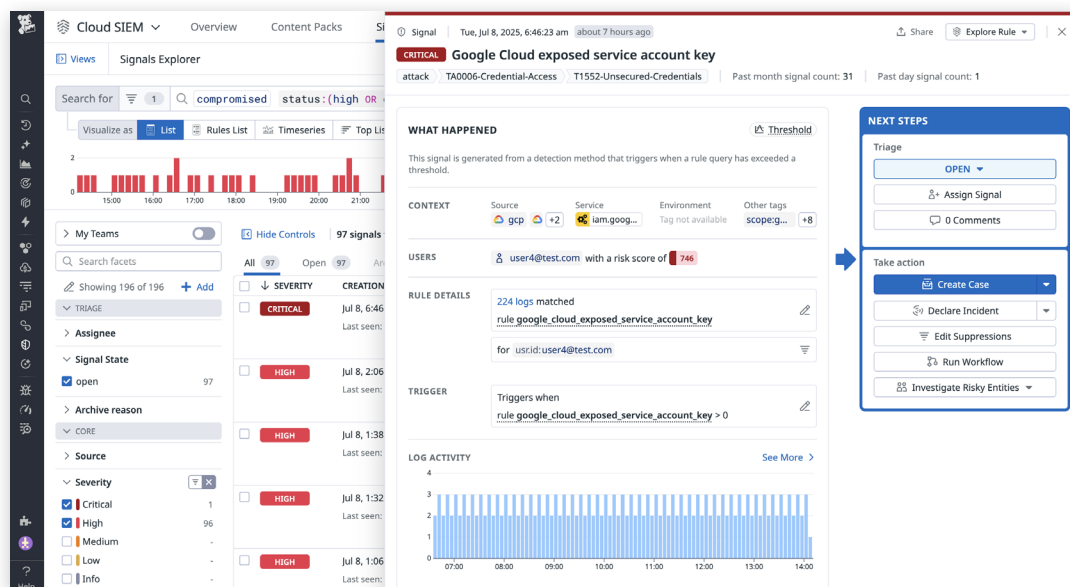
OTHER TACTICS AND TECHNIQUES

In addition to the three tactics that we covered, others can be mapped to AI infrastructure, including:

- **Reconnaissance:** Techniques such as **active scanning** enable an attacker to gather information about available infrastructure resources and networks.
- **Resource development:** With techniques such as **establish accounts** and **acquire infrastructure**, attackers create new cloud user or service accounts to accomplish their goals.

Detect and minimize the risks that lead to AI abuse

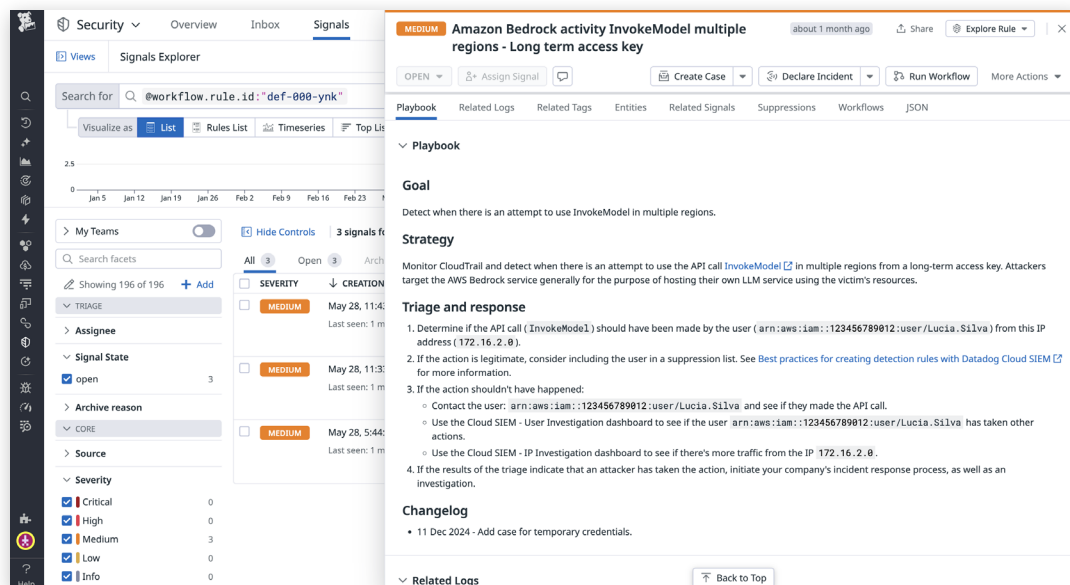
While these tactics are only a few of the ways attackers can target AI infrastructure, they highlight issues that are commonly seen in both cloud and AI security: misconfigured infrastructure and mismanaged credentials. Detecting these risks starts with the same strategies that are used for securing cloud environments. These include scanning them for **old or unused credentials** or **monitoring logs** for issues, such as compromised account keys for cloud services, as seen in the following Datadog Cloud SIEM signal:



In cases like these, **minimizing the use of long-lived credentials** can help reduce the risk of compromised credentials.

Unusual access patterns can also surface the early signs of an attacker attempting to abuse AI infrastructure via the initial access or credential access tactics. It can also capture malicious activity post-compromise, such as the API calls described in the previously mentioned LLM jacking scenario.

For example, if an attacker executes legitimate `InvokeModel` calls to Amazon Bedrock with unusual call parameters, Datadog Cloud SIEM will flag these calls and provide details about their source and parameters, as seen in the following screenshot:



Datadog Cloud SIEM will also **automatically flag other calls to Amazon Bedrock** that could indicate that an attacker is using the discovery tactic to find available LLMs.

Minimize threats to AI infrastructure

In this chapter, we looked at common threats targeting AI infrastructure and how to detect them in cloud environments. Datadog Cloud SIEM and Cloud Security can help you identify compromised credentials and infrastructure as well as abnormal activity. Check out our documentation to learn more about [Datadog's security offerings](#). Or, read on to learn how [attackers abuse AI artifacts in supply chains](#). If you don't already have a Datadog account, you can sign up for a [free 14-day trial](#).



Abusing supply chains: How poisoned models, data, and third-party libraries compromise AI systems



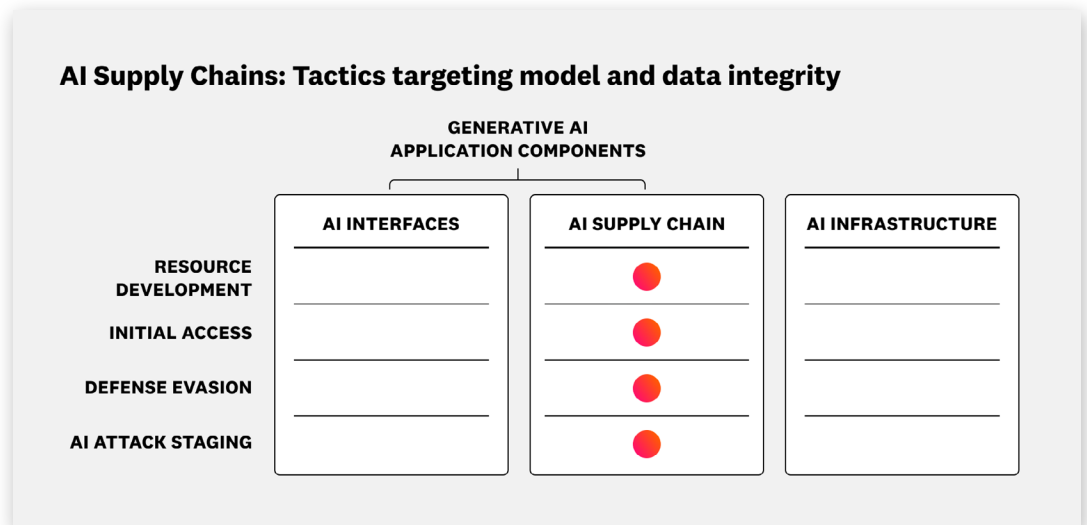
Mallory Mooney

The AI ecosystem is rapidly changing, and with this growth comes unique challenges in securing the infrastructure and services that support it. In Part 1 of this series, we explored [how attackers target the underlying resources that host and run AI applications](#), such as cloud infrastructure and storage. In this chapter, we'll look at threats that affect AI-specific resources in supply chains, which are the software and data artifacts that determine how an AI service operates. Supply chain artifacts can include training datasets, pre-trained models, and third-party AI libraries. We'll map the threats that target supply chain components to [MITRE's Adversarial Threat Landscape for Artificial Intelligence Systems \(ATLAS\) tactics and techniques](#), which summarize why and how attackers target AI components. We'll also look at how to [detect and minimize these threats](#).

Common attacks that target AI artifacts in supply chains

Revisiting our analogy from Part 1, if the infrastructure for generative AI is the highway, then supply chains provide the transportation and routes. They use the infrastructure—cloud storage, compute instances, and servers—to deliver functionality to users. In many cases, organizations integrate both in-house and third-party supply chain artifacts into their AI applications. This approach gives them greater flexibility in how and what they develop, but it also means they have less control over application

security. Consequently, organizations may unknowingly include malicious third-party artifacts, such as poisoned training data or models, into their applications' supply chains. These risks enable attackers to execute specific tactics, such as developing and staging attacks against supply chains, gaining [initial access](#) to AI artifacts, and [avoiding detection](#). As seen in the following diagram, visualizing where these tactics occur can help you prioritize monitoring and implement security controls that cover the areas that attackers target the most.



DEVELOPING AND STAGING AN ATTACK

The early stages of many attacks involve attackers building and testing tools that they will later use to compromise an application. Because this activity often occurs outside the organization's visibility, it leaves no immediate signs of compromise. The value in understanding these early stages is that they offer insight into how attackers might repurpose available assets in ways you may not expect.

For example, [resource development](#) involves an attacker gathering the resources they need to execute other attack phases. During this stage, attackers may attempt to [acquire public AI artifacts](#) from organizations in order to publish their own malicious artifacts, such as poisoned [datasets](#), [training data](#), and [models](#), on popular repositories and package registries.

In one case, security researchers were able to [publish packages under hallucinated names](#) to the Python Package Index (PyPI). AI-generated code like this is a [growing risk to the supply chain](#), as it leads teams to pull actively malicious dependencies for their applications.

In addition to developing resources in the early stages, attackers may [prep and stage their attacks](#) on target AI models to test their readiness. Examples include [creating and training proxy models](#) and [crafting data to feed to them](#). This enables them to ensure that their attacks will work against legitimate models.

Because these tactics often occur offline before a direct attack is executed, it's difficult to anticipate how an attacker will target a part of the supply chain. This gap in visibility into early-stage attacks highlights the need for a defense-in-depth approach to risk detection and remediation. This is especially important as organizations increasingly rely on AI coding tools but may not thoroughly vet created dependencies, which can later become attack vectors.

GAINING INITIAL ACCESS

Like with AI infrastructure, attackers will use **initial access techniques** to exploit the supply chain. This tactic builds on the resource development and staging phases, using the artifacts attackers have collected or prepared to gain entry.

One example of this attack path involved a **researcher creating a believable but unverified organization account** on a popular public model repository. Employees unknowingly joined this fake organization and uploaded private AI models, which the researcher then had full read and write access to. The researcher **compromised an available model** by embedding a command and control (C2) server and republishing it on the account. When an employee used the compromised model, the embedded malware enabled the researcher to control the victim's machine via the established C2 channel. In this example, attackers used supply chain artifacts hosted on public repositories, where they blended in with legitimate packages and accounts. This highlights the need to **maintain visibility into supply chain artifacts**, especially as the AI ecosystem evolves.

AVOIDING DETECTION

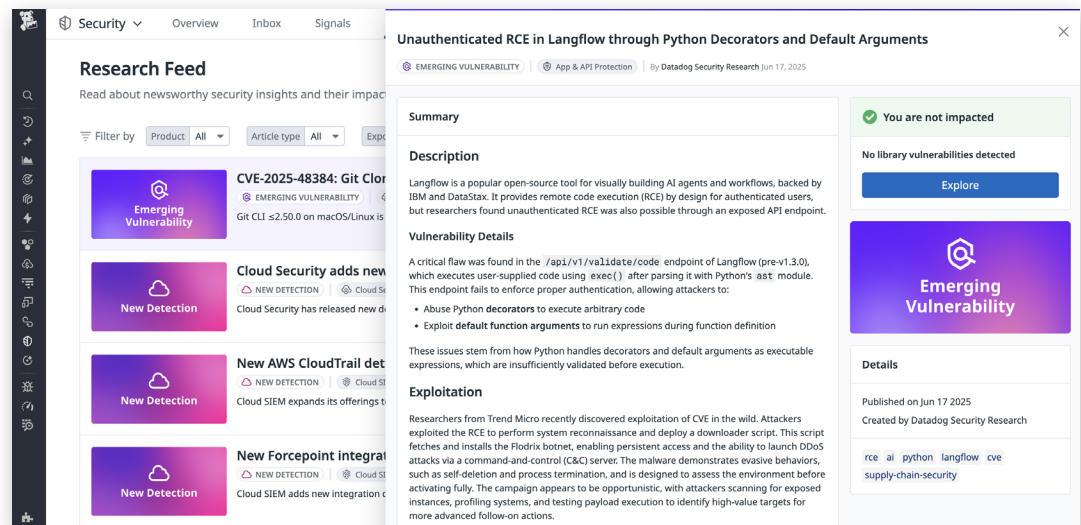
Once an attacker has access to an environment, they will attempt to **avoid detection** by security monitoring. In supply chains, this can look like attackers publishing corrupted models that successfully evade available scanning tools, such as Hugging Face's security scanner. In one incident, researchers discovered malicious AI models on the Hugging Face repository that had **evaded detection** by exploiting a **known risk with Python's pickle files**.

Detect and minimize the inherent risks in supply chains

We've only covered a few of the ways attackers exploit supply chain vulnerabilities. These examples are part of a much larger and evolving set of tactics targeting its artifacts. Attackers are increasingly taking advantage of the complexity of the supply chain ecosystem by using both direct and indirect methods to manipulate models and their architecture and data.

Because these attacks span the entire supply chain, organizations need to be proactive in how they monitor the artifacts they use. This is especially true considering the risk of unknowingly pulling in potentially vulnerable third-party AI packages, such as the [popular Python AI package Langflow](#).

Understanding where vulnerabilities exist is a critical starting point for AI security monitoring. Datadog Security Research recently published an [emerging vulnerability notice](#) that highlights risks associated with the Langflow package. The notice also included information about where [Datadog Code Security](#) detected this risk within a particular environment:



[Vulnerability scanning](#) using tools like Code Security and [GuardDog](#) provides readily available visibility into security risks with third-party libraries, which attackers often take advantage of as part of their initial access and defense evasion tactics.

In addition to knowing where vulnerabilities exist in library code, proactive security monitoring requires an understanding of how attackers find and use AI libraries and data. This enables organizations to set guardrails around how both internal and external users access, share, maintain, and build supply chain artifacts. MITRE offers the following recommendations:

- [Create authorization controls for using internal model registries, production models, and training data](#): This step limits an attacker's capabilities if they gain access to an organization's environment, such as [through its AI infrastructure](#).
- [Restrict which information about production model architecture and data is publicly available](#): Because attackers use this data to develop their resources and stage AI attacks, limiting what's available prevents them from taking advantage of it.
- [Regularly sanitize training data](#): Attackers attempt to manipulate models by publishing poisoned training data on public registries. Sanitizing any

publicly available data or datasets before use reduces the likelihood of attackers poisoning internal models.

- **Routinely check AI model behavior and data:** As with sanitizing training data, validating that AI models are operating as expected can help surface signs of compromise, such as poisoning.
- **Code signing:** This step ensures that supply chain artifacts are verified with digital signatures, which prevents organizations from integrating unauthorized models and other software into their applications.

Minimize threats to AI artifacts in supply chains

In this chapter, we looked at common threats that target the supply chain and how to detect and mitigate them. Check out our documentation to learn more about [Datadog's security offerings](#), such as Datadog Code Security. Or, read on to explore [the ways attackers abuse AI user interfaces](#) in Part 3 of this series. If you don't already have a Datadog account, you can sign up for a [free 14-day trial](#).



Abusing AI interfaces: How prompt-level attacks exploit LLM applications



Mallory Mooney

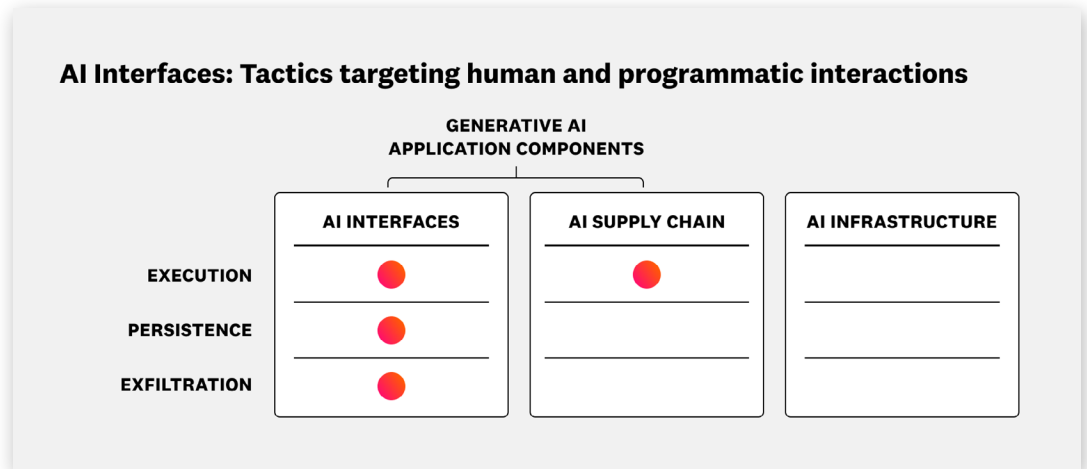
In Parts 1 and 2 of this series, we looked at how attackers get access to and take advantage of the infrastructure and supply chains that shape generative AI applications. In this chapter, we'll discuss AI interfaces, which we define as the entry points and logic that determine how a user interacts with an AI application. These elements can include chat interfaces, such as AI assistants, and API endpoints for supporting services. We'll map the threats that target AI interfaces to [MITRE's Adversarial Threat Landscape for Artificial Intelligence Systems \(ATLAS\) tactics and techniques](#), which summarize why and how attackers target various components of generative AI applications. We'll also look at how to [detect and minimize these threats](#).

Common attacks that target AI interfaces

Continuing with our analogy from Parts 1 and 2, if AI infrastructure serves as the highway and supply chains are the delivery systems, AI interfaces are the entrance ramps that enable users to access an application's functionality. This layer relies on user input, which is impossible to fully predict. This means that attackers can take advantage of the flexibility and unpredictability of natural language in order to craft malicious prompts that blend in with real user interactions.

This variability makes AI interfaces a key target for many post-compromise tactics, such as [hijacking LLM behavior](#), [maintaining control](#) over an

interface, and [leaking data](#). The following diagram shows how tactics like these concentrate at the AI interface, but they can also expand an attacker's reach by exploiting interface dependencies, such as supply chain artifacts. By mapping where these attacks occur, you can identify gaps in security monitoring and response.



Embedding and executing harmful code

For user-facing interfaces, such as AI assistants, attackers may use [execution](#) to run harmful code embedded in AI artifacts, such as those used in supply chains. This tactic is typically used as part of a broader attack, supporting other tactics such as **initial access** and [data exfiltration](#).

Commonly used techniques during this stage include both [direct and indirect prompt injections](#), which attempt to hijack LLM behavior. In one instance, an attacker successfully [leaked private SQL database tables via a prompt injection attack](#).

Researchers used a similar approach to the one in our [previously mentioned M365 Copilot scenario](#), which combined several tactics, including initial access and execution. In this instance, they used an indirect prompt injection to change LLM behavior every time an employee used a poisoned RAG entry. These examples demonstrate how the execution tactic uses AI interfaces to gain access to other parts of the application's infrastructure and supply chains. [Detecting prompt injections](#) can serve as a first line of defense and stop these attacks before they escalate.

MAINTAINING A FOOTHOLD

Once attackers have access to an AI application or enough information about how to manipulate it, they can attempt to maintain a foothold via [persistence](#). This enables attackers to continue their attacks against a

particular application, even if an organization takes steps to limit their access. For AI interfaces, that can look like manipulating a model or its context. In one case, attackers manipulated a [Twitter chatbot's responses](#) by using specific, coordinated prompts. Because the chatbot learned from user input, the prompts gradually degraded its responses.

This example highlights the complexity of managing user input, where organizations need the ability to reliably identify and prioritize harmful input. One toxic prompt on its own might not signal an attack, for example, but a pattern of that type of input is worth monitoring.

Exfiltrating data

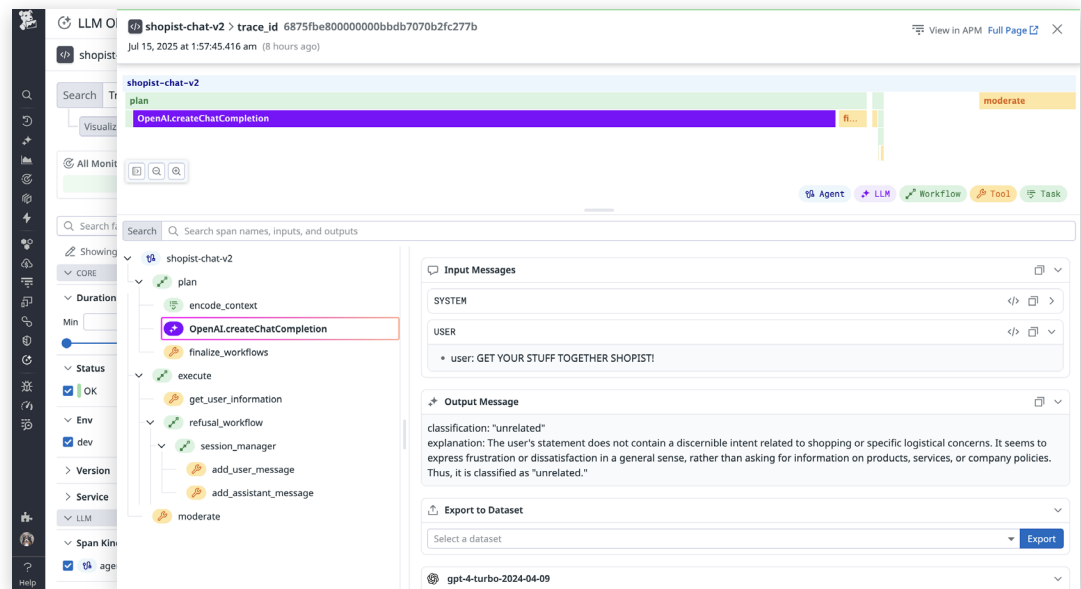
One of the primary goals of an attacker is to find and [steal sensitive data](#). This tactic often starts at the interfaces that enable attackers to interact with a model. LLMs in particular are prone to [data leakage](#), especially if the application using them is deployed without appropriate guardrails for user input. One way that attackers target LLMs and their interfaces is through data leakage, which involves manipulating a model to reveal sensitive data that it has memorized or has access to via API services or other tools.

In one exercise, researchers were able to use [a malicious, self-replicating prompt with a RAG-based assistant](#). The prompt injection, which was part of the execution tactic, included instructions for the assistant to provide sensitive user data in its response.

Detect and minimize the inherent risks in AI interfaces

We've covered only a few of the ways attackers exploit interfaces, but they highlight the fact that prompts are one of the primary security concerns for generative AI applications. Because prompts are a new and evolving attack surface, organizations need to create new detection patterns that focus on unpredictable user-to-model interactions and model output, such as sensitive data exposure. Tracking these patterns is especially critical as organizations scale their AI applications to handle more user requests.

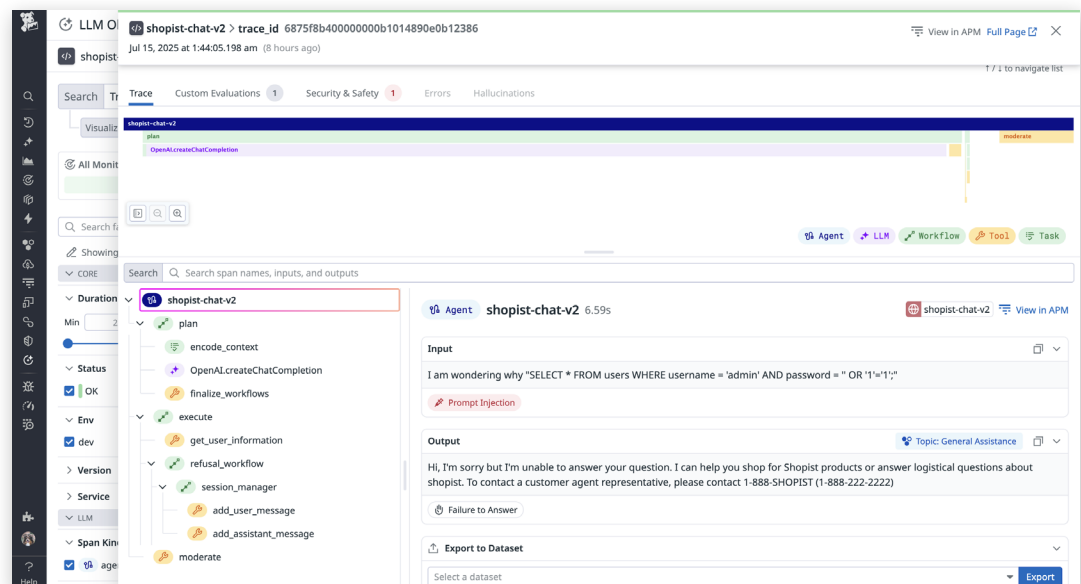
Monitoring for toxic prompts, for example, can give organizations visibility into instances where attackers are attempting to [maintain their influence over LLM behavior](#). In the following example evaluation, Datadog LLM Observability automatically [flags toxic content](#) submitted to a chatbot:



To help prevent an end user from succeeding in manipulating a model, organizations can **sanitize prompt inputs** before the model receives them. This can surface suspicious patterns more efficiently than allowing all input. In addition, **rate limiting** inputs throttles the volume of prompts that attackers may attempt to generate.

Monitoring for more direct attacks against AI interfaces, such as prompt injections, is also an important part of keeping generative AI applications and their data secure. Injection attacks can show up as unusual prompt patterns, such as prompts with long instructions, nested logic, and out-of-scope queries. When successful, these attacks can leak sensitive data, including credentials and personally identifiable information (PII), as output for the attacker.

In the following evaluation, Datadog LLM Observability automatically flags an attempt to exfiltrate data from an SQL database via a prompt injection:



In cases like these, [controlling model permissions and which APIs, tools, and data they can call or access](#) will help minimize the risk of successful prompt injections, such as those mentioned as part of the [exfiltration tactic](#). [Implementing output filtering](#) can also remove sensitive data and clean degraded responses before a model generates output.

For an in-depth look at how to detect and respond to these tactics, check out our [guide to monitoring for prompt injection attacks](#).

Protect your generative AI applications from abuse

In this chapter, we looked at common threats that target AI interfaces, such as chatbots and assistants, and how to detect and prevent them. Check out our documentation to learn more about [Datadog's security offerings](#), such as Datadog Code Security. If you don't already have a Datadog account, you can sign up for a [free 14-day trial](#).



Secure your AI Applications with Datadog

See how Datadog helps protect against threats that target the infrastructure, supply chain, and AI interfaces of your AI applications.

TRY DATADOG FOR FREE